

百色学院

《网络接口编程》课程设计

博客管理系统

系统设计文档



项目名称：博客管理系统 (Blog Management System)

课程名称：网络接口编程

技术栈：Spring Boot + Spring MVC + MyBatis + MySQL + Bootstrap

版本：1.0.0

日期：2026年5月

目 录

1、项目背景与需求分析

1.1 项目背景

1.2 需求分析概述

1.3 功能性需求

1.4 非功能性需求

2、系统功能模块划分

2.1 系统总体功能模块

2.2 各模块详细说明

2.3 用户角色与用例

3、数据库设计

3.1 概念设计（ER图）

3.2 逻辑设计（表结构）

3.3 表关系说明

4、系统架构设计

4.1 系统总体架构

4.2 MVC分层架构

4.3 请求处理流程

4.4 代码组织结构

5、技术选型说明

5.1 后端技术栈

5.2 前端技术栈

5.3 开发工具与环境

一、项目背景与需求分析

1.1 项目背景

随着互联网技术的快速发展和信息传播方式的深刻变革，博客作为一种重要的网络信息发布与分享平台，在现代社会中扮演着越来越重要的角色。无论是个人用户记录生活感悟、分享专业知识，还是企业机构进行品牌宣传、技术沉淀，博客系统都提供了便捷高效的解决方案。

目前市面上虽然有 WordPress、Typecho 等成熟的博客平台，但这些平台功能繁杂、定制成本高，对于学习 Java EE 企业级开发技术的学生而言，从零开始构建一个轻量级博客管理系统，能够深入理解 Web 应用从后端到前端的完整开发流程，掌握 Spring Boot、Spring MVC、MyBatis 等主流框架的实际应用。

本课题旨在设计并实现一个功能完整、结构清晰的博客管理系统。系统采用 B/S 架构，基于 Spring Boot 框架进行后端开发，使用 MyBatis 作为数据持久层框架，MySQL 作为数据库存储引擎，前端采用 Bootstrap 框架构建响应式用户界面。通过本项目的开发，综合检验对网络接口编程课程所学知识的掌握程度，包括 RESTful API 设计、数据持久化、前后端交互、权限控制等核心技能。

该系统面向两类用户群体：普通用户和管理员。普通用户可以浏览文章、发表评论，管理员则在此基础上拥有文章管理、分类管理和用户管理等高级权限。系统要求具备完整的用户认证流程、规范的数据库设计、良好的页面交互体验以及稳定的运行性能。

1.2 需求分析概述

经过对博客系统业务场景的深入分析，将系统需求划分为功能性需求和非功能性需求两大部分。功能性需求描述系统应提供的具体业务功能，非功能性需求则关注系统的性能、安全、可用性等方面。

1.3 功能性需求

模块	编号	功能名称	功能描述	用户类型
用户模块	F-001	用户注册	用户填写用户名、邮箱、密码完成注册	所有用户
	F-002	用户登录	输入用户名和密码进行身份认证，返回 JWT Token	所有用户
	F-003	个人信息管理	查看和修改个人基本资料	已登录用户
	F-004	退出登录	清除登录状态，退出系统	已登录用户
文章模块	F-005	发布文章	填写标题、正文、分类等发布新文章	已登录用户
	F-006	编辑文章	修改已发布文章的内容	作者本人
	F-007	删除文章	删除指定的文章	作者/管理员
	F-008	文章列表浏览	分页浏览所有文章，支持按分类筛选和模糊搜索	所有用户
	F-009	查看文章详情	查看文章完整内容和评论	所有用户
评论模块	F-010	发表评论	对指定文章发表评论	已登录用户
	F-011	评论列表查看	查看文章下的所有评论	所有用户
	F-012	删除评论	删除不当评论	作者/管理员

分类模块	F-013	创建分类	新增文章分类（如技术、生活、随笔等）	管理员
	F-014	编辑分类	修改分类名称和描述	管理员
	F-015	删除分类	删除指定分类	管理员
	F-016	分类列表查看	查看所有分类及对应文章数量	所有用户

1.4 非功能性需求

需求类别	需求描述	具体指标
性能需求	页面响应时间	核心业务页面响应时间不超过3秒
	并发支持	支持至少50个用户同时在线访问
	数据库查询效率	带索引查询响应时间不超过1秒
安全需求	用户认证	使用 JWT Token 进行身份认证，密码使用 SHA-256 加密存储
	权限控制	基于拦截器实现接口级权限校验，Cookie+Header 双通道 Token 验证
	数据安全	MyBatis 预编译防止 SQL 注入
可用性需求	界面设计	Bootstrap 5 响应式布局，兼容 PC/移动端
	错误处理	@ControllerAdvice 全局统一异常处理，友好提示
	可维护性	MVC 分层架构，代码结构清晰，注释规范

二、系统功能模块划分

2.1 系统总体功能模块

博客管理系统按业务功能划分为以下四大核心模块：

模块	子功能	说明
用户模块	注册 / 登录 / 退出	用户身份认证与会话管理（JWT）
	个人信息管理	用户查看和修改个人资料
文章模块	发布文章	创建新博客文章（含标题、正文、分类、封面图）
	编辑 / 删除文章	对已发布文章的维护操作（仅作者或管理员）
	文章列表浏览	分页展示文章，支持按分类筛选和模糊搜索
	文章详情查看	查看文章完整内容，同时展示相关评论
评论模块	发表评论	登录用户对文章进行评论互动
	评论列表查看	查看文章下按时间排序的所有评论
	删除评论	文章作者或管理员可删除评论
分类模块	分类管理	管理员新增、编辑、删除文章分类
	分类展示	展示所有分类列表

2.2 各模块详细说明

（1）用户模块

用户模块负责系统的身份认证和用户信息管理。新用户通过注册功能创建账户，系统对密码进行 SHA-256 加密处理，确保用户信息安全。注册成功后，用户可通过登录功能输入用户名和密码进行身份认证，认证通过后服务器返回 JWT 令牌（Token），前端将 Token

同时存储在 localStorage 和 Cookie 中，以同时支持 AJAX 请求和页面导航的认证需求。已登录用户可在个人信息页面查看和修改个人资料。退出登录时，前端清除本地存储的 Token 和 Cookie。

（2）文章模块

文章模块是博客系统的核心业务模块。已登录用户可以创建新文章，填写标题、正文内容、选择分类以及上传封面图片。文章发布后，所有访客（含未登录用户）可在首页浏览文章列表，列表支持分页展示、按分类筛选以及按标题或内容的关键字模糊搜索。用户点击文章标题可进入详情页查看完整内容和评论。文章作者和管理员拥有编辑和删除文章的权限。

（3）评论模块

评论模块为博客系统提供了互动功能。已登录用户可在文章详情页发表评论，评论内容会实时显示在该文章下方的评论列表中。评论列表按时间倒序排列，方便用户查看最新评论。文章作者和系统管理员有权删除不当评论，维护社区秩序。评论的发表和删除均采用 Ajax 异步请求方式，提升用户体验。

（4）分类模块

分类模块用于对博客文章进行归类管理。管理员可以创建、编辑和删除文章分类，例如"技术博客"、"生活随笔"、"学习笔记"等。每篇文章必须属于一个分类，分类列表展示所有分类名称。用户在首页可通过点击分类标签快速筛选该分类下的所有文章。

2.3 用户角色与用例

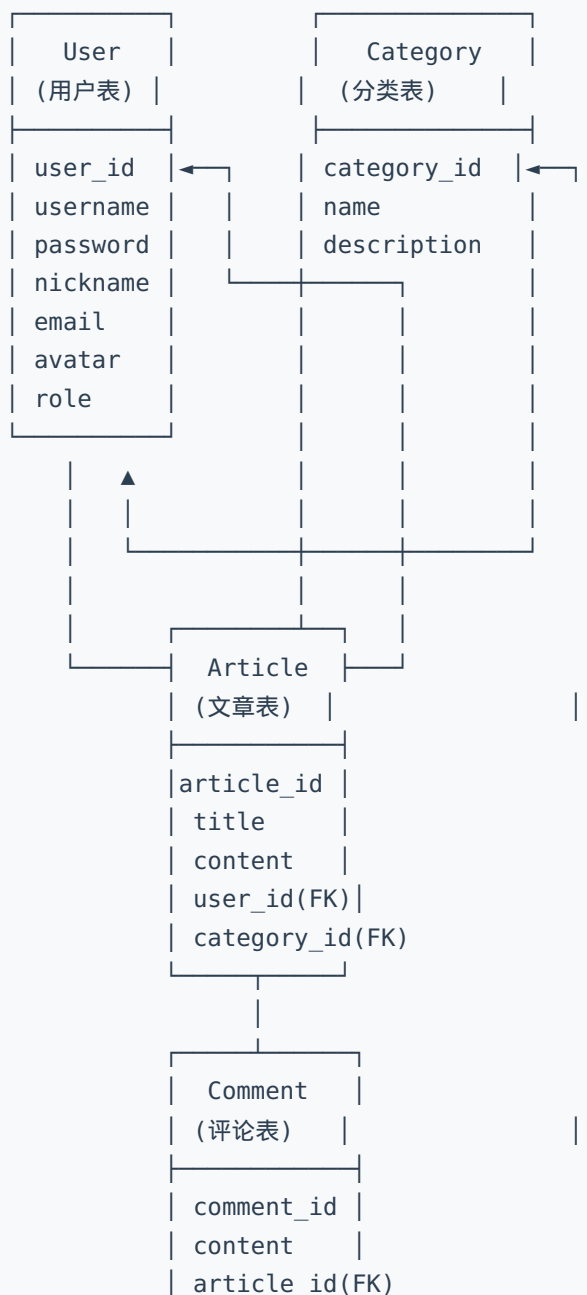
系统涉及两类角色：普通用户（User）和管理员（Admin）。

角色	可用功能
普通用户	注册账号、登录系统、浏览文章列表、查看文章详情、搜索文章、按分类筛选、发表评论、管理自己的文章（新增/编辑/删除）
管理员	包含普通用户所有权限 + 管理所有文章、管理所有评论、管理文章分类（增/删/改）

三、数据库设计

3.1 概念设计（ER 图）

根据需求分析，系统涉及的核心实体包括：用户（**User**）、文章（**Article**）、评论（**Comment**）和分类（**Category**）。各实体之间的关系如下：



| user_id(FK) |

实体关系说明：

- User (1) → Article (N) - 一个用户可以发布多篇文章
- User (1) → Comment (N) - 一个用户可以发表多条评论
- Category (1) → Article (N) - 一个分类下可以有多篇文章
- Article (1) → Comment (N) - 一篇文章可以有多条评论

3.2 逻辑设计（表结构）

数据库名：blog_db，字符集：utf8mb4。

（1）user（用户表）

字段名	类型	约束	说明
user_id	INT(11)	主键，自增	用户ID
username	VARCHAR(50)	非空，唯一	用户名
password	VARCHAR(255)	非空	密码（SHA-256 加密）
nickname	VARCHAR(50)	非空	用户昵称
email	VARCHAR(100)	非空，唯一	邮箱
avatar	VARCHAR(255)	可空	头像URL
role	VARCHAR(20)	默认 'user'	角色：user/admin
status	TINYINT(4)	默认 1	状态：1启用/0禁用
create_time	DATETIME	默认当前时间	创建时间
update_time	DATETIME	自动更新	更新时间

（2）category（分类表）

字段名	类型	约束	说明
-----	----	----	----

category_id	INT(11)	主键，自增	分类ID
name	VARCHAR(50)	非空，唯一	分类名称
description	VARCHAR(255)	可空	分类描述
sort_order	INT(11)	默认 0	排序序号
create_time	DATETIME	默认当前时间	创建时间
update_time	DATETIME	自动更新	更新时间

(3) article (文章表)

字段名	类型	约束	说明
article_id	INT(11)	主键，自增	文章ID
title	VARCHAR(200)	非空	文章标题
content	TEXT	非空	文章正文
summary	VARCHAR(500)	可空	文章摘要
cover_image	VARCHAR(255)	可空	封面图片URL
user_id	INT(11)	非空，外键 → user	作者ID
category_id	INT(11)	非空，外键 → category	分类ID
status	TINYINT(4)	默认 1	状态：1发布/0草稿
view_count	INT(11)	默认 0	浏览次数
create_time	DATETIME	默认当前时间	创建时间
update_time	DATETIME	自动更新	更新时间

索引：idx_user_id (user_id) ， idx_category_id (category_id) ， idx_create_time (create_time) ， ft_title_content (FULLTEXT 全文索引)

(4) comment (评论表)

字段名	类型	约束	说明
comment_id	INT(11)	主键，自增	评论ID
content	VARCHAR(500)	非空	评论内容
article_id	INT(11)	非空，外键 → article	所属文章ID
user_id	INT(11)	非空，外键 → user	评论者ID
create_time	DATETIME	默认当前时间	评论时间

3.3 表关系说明

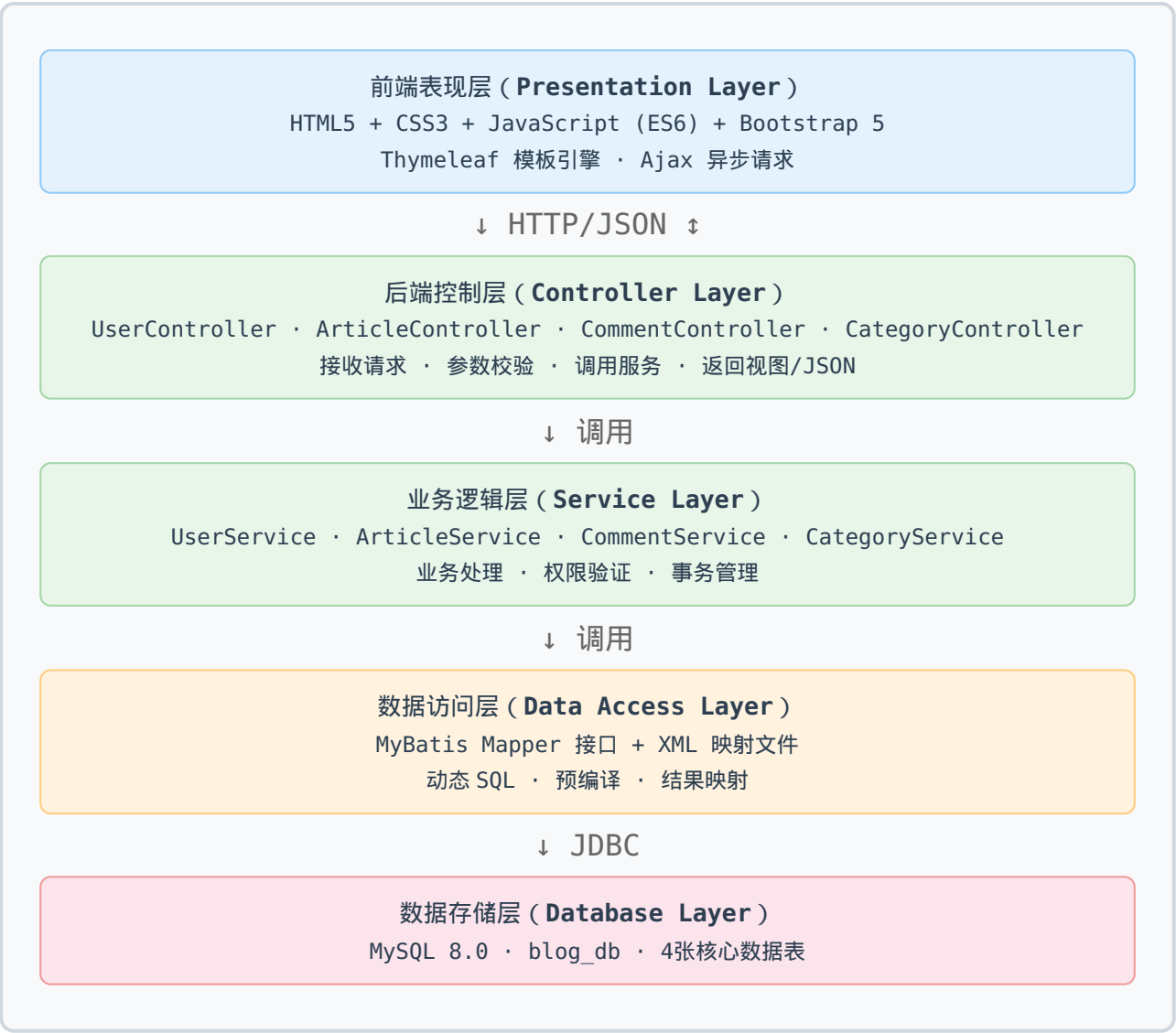
四张表通过外键约束形成完整的关联关系：

关系	外键	说明
User 1 → N Article	article.user_id → user.user_id	一个用户可以发布多篇文章
User 1 → N Comment	comment.user_id → user.user_id	一个用户可以发表多条评论
Category 1 → N Article	article.category_id → category.category_id	一个分类下可有多篇文章
Article 1 → N Comment	comment.article_id → article.article_id	一篇文章可有多条评论

四、系统架构设计

4.1 系统总体架构

博客管理系统采用基于 B/S (Browser/Server) 模式的分层架构设计。系统整体分为表现层、业务逻辑层和数据访问层，各层职责清晰、耦合度低，便于开发和维护。



4.2 MVC 分层架构

系统严格遵循 Spring MVC 的分层设计思想，各层职责如下：

层次	组件	包路径	职责说明
Controller 层	UserController	controller	处理用户注册、登录等请求
	ArticleController	controller	文章增删改查、页面导航
	CommentController	controller	评论发表与删除
	CategoryController	controller	分类管理（仅管理员）
Service 层	UserService + Impl	service/impl	用户认证、密码加密
	ArticleService + Impl	service/impl	文章 CRUD、分页查询
	CommentService + Impl	service/impl	评论业务逻辑
	CategoryService + Impl	service/impl	分类管理业务逻辑
Mapper 层	UserMapper + XML	mapper	用户数据持久化
	ArticleMapper + XML	mapper	文章数据持久化（含 JOIN）
	CommentMapper + XML	mapper	评论数据持久化
	CategoryMapper + XML	mapper	分类数据持久化
基础设施	JwtUtil	util	JWT Token 生成与验证
	Result	util	统一 API 响应封装
	LoginInterceptor	interceptor	请求认证拦截（Header + Cookie）
	GlobalExceptionHandler	handler	全局异常统一处理

4.3 请求处理流程

一次典型的请求处理流程如下（以“查看文章详情”为例）：

步骤1：浏览器点击文章标题 → 发送 GET 请求 /api/articles/detail-page/1
 步骤2：DispatcherServlet 前端控制器拦截请求
 步骤3：根据 URL 匹配到 ArticleController 的 detailPage() 方法
 步骤4：ArticleService 调用 ArticleMapper 查询数据库（含 User 和 Category 的 JOIN）

步骤5：MyBatis 执行预编译 SQL，返回查询结果

步骤6：Controller 将数据封装为 Model，渲染 Thymeleaf 模板

步骤7：浏览器响应 HTML 页面，呈现给用户

4.4 项目代码结构

```
blog-system/
├─ pom.xml                # Maven 项目配置
├─ sql/init.sql           # 数据库建表脚本
└─ src/main/
    ├─ java/com/blog/
    │   ├─ BlogApplication.java    # Spring Boot 启动类
    │   ├─ config/WebMvcConfig.java # MVC 配置 + 拦截器注册
    │   ├─ controller/             # 4 个控制器
    │   ├─ service/ + impl/         # 4 个 Service 接口 + 实现
    │   ├─ mapper/                 # 4 个 Mapper 接口
    │   ├─ entity/                 # 4 个实体类
    │   ├─ dto/                    # 4 个数据传输对象
    │   ├─ interceptor/            # JWT 登录拦截器
    │   ├─ handler/                # 全局异常处理
    │   └─ util/                   # JWT 工具 + 统一响应
    └─ resources/
        ├─ application.yml         # 应用配置
        ├─ mapper/                 # 4 个 MyBatis XML 映射
        ├─ static/css/ + js/       # 自定义样式和脚本
        └─ templates/              # 10 个 Thymeleaf 页面
```

五、技术选型说明

5.1 后端技术栈

技术组件	版本	选型理由
Java	JDK 17	LTS 长期支持版本，支持 Records、Sealed Class 等现代语言特性，性能优异
Spring Boot	3.2.0	自动装配简化配置，内置 Tomcat 服务器一键启动，与 Spring MVC、MyBatis 无缝集成
Spring MVC	6.1.1	@RestController 注解支持 RESTful API 开发，HandlerInterceptor 实现权限拦截，@ControllerAdvice 统一异常处理
MyBatis	3.0.3	手写 SQL 优化灵活，支持动态 SQL 标签（if/where/foreach），自动结果映射，预编译机制防 SQL 注入
MySQL	8.0	业界最流行的开源关系型数据库，性能优越，与 Java/MyBatis 集成方案成熟
Maven	3.9+	标准化依赖管理和项目构建，提供 clean/compile/test/package 等完整生命周期

核心依赖清单

依赖	用途
spring-boot-starter-web	Spring Boot Web 核心（含内嵌 Tomcat）
spring-boot-starter-thymeleaf	服务端模板引擎
mybatis-spring-boot-starter	MyBatis 与 Spring Boot 集成
mysql-connector-j	MySQL JDBC 驱动
jjwt-api / jjwt-impl / jjwt-jackson	JWT Token 生成与解析

lombok	简化实体类代码（@Data、@AllArgsConstructor 等）
pagehelper-spring-boot-starter	MyBatis 分页插件
spring-boot-starter-validation	参数校验（@Valid、@NotBlank 等）

5.2 前端技术栈

技术组件	说明
HTML5	语义化标签（header/nav/article/section）使页面结构清晰
CSS3	Flexbox/Grid 布局 + Transition/Animation 动画
JavaScript (ES6+)	模块化语法 + async/await 异步请求
Bootstrap 5.3	响应式 12 栅格系统，丰富的组件库（导航栏、卡片、模态框等），跨浏览器兼容
Thymeleaf	与 Spring Boot 天然集成，支持在 HTML 中直接访问后端数据

5.3 RESTful API 接口一览

方法	路径	说明	权限
POST	/api/users/login	用户登录	公开
POST	/api/users/register	用户注册	公开
GET	/api/users/me	获取当前用户信息	登录
PUT	/api/users/profile	更新个人信息	登录
POST	/api/users/logout	退出登录	公开
GET	/api/articles/list	文章列表（分页/分类/搜索）	公开
GET	/api/articles/detail/{id}	文章详情	公开

POST	/api/articles	发布文章	登录
PUT	/api/articles/{id}	更新文章	登录
DELETE	/api/articles/{id}	删除文章	登录
GET	/api/comments/list/{articleId}	评论列表	公开
POST	/api/comments	发表评论	登录
DELETE	/api/comments/{id}	删除评论	登录
GET	/api/categories/list	分类列表	公开
POST	/api/categories	创建分类	管理员
PUT	/api/categories/{id}	更新分类	管理员
DELETE	/api/categories/{id}	删除分类	管理员

5.4 开发工具与环境

工具	用途
IntelliJ IDEA / VS Code	Java 项目开发 IDE
MySQL Workbench / Navicat	数据库可视化管理
Postman / curl	RESTful API 接口调试
Git + GitHub	版本控制与代码管理
Chrome DevTools	前端调试与性能分析
Maven	项目构建与依赖管理